

Term Project Report

Parallel N-body Gravitational Simulation:
Peer-to-Peer MPI Decomposition with Leapfrog Integration

Student Name: Sean Balbale

Course: CPSC 375: High-Performance Computing

Instructor: Peter Yoon

Date: April 26, 2026

1 Introduction

The N -body problem asks how N particles evolve under pairwise gravitational forces: every particle interacts with every other, and the system state is advanced through time by a numerical integrator. The canonical direct-sum algorithm evaluates all $N(N - 1)/2$ pairwise interactions per timestep, yielding $O(N^2)$ complexity per step. This quadratic cost makes large-scale N -body simulation one of the defining benchmarks of computational physics; for $N = 9,600$ particles and 10 timesteps, a serial execution requires on the order of 10^{10} floating-point operations.

The structure of the computation maps naturally to distributed-memory parallelism. Force evaluation on particle i depends only on the *current* positions of all other particles — not on their forces — so the work partitions cleanly across MPI ranks without synchronization during force accumulation. Each rank owns a contiguous block of N/P particles, computes the full pairwise forces on its block using the replicated global position array, and then participates in an `MPI_Allgatherv` collective to synchronize updated positions before the next timestep. The per-rank compute load scales as $O(N^2/P)$; the communication load scales as $O(N)$ per step, making the communication-to-computation ratio proportional to P/N .

This project implements a fully parallel N -body gravitational simulator in C using MPI, integrated with the leapfrog (Störmer-Verlet) scheme for energy-conserving numerical time integration. Strong and weak scaling studies executed on the Three Idiots cluster — three nodes interconnected over a 10.0.0.0/24 TCP fabric, totaling 24 available MPI processes — quantify how effectively the implementation exploits additional hardware. Amdahl’s law is applied to the strong scaling data to extract the implied serial fraction and theoretical maximum speedup for each problem size.

2 Problem Description

The specific goals of this project are:

- To implement a correct, parallel N -body gravitational simulator in C using MPI, using a fully symmetric peer-to-peer decomposition in which all P ranks perform force computation.
- To design a communication pattern based on `MPI_Allgatherv` that synchronizes global particle positions after each timestep with minimal per-step overhead.
- To conduct a strong scaling study at three particle counts ($N = 2400, 4800, 9600$) and measure speedup $S(P) = T(1)/T(P)$ and parallel efficiency $E(P) = S(P)/P$ across $P = 1$ to $P = 24$.
- To conduct a weak scaling study holding particles per process constant at 1000 and quantify how measured wall time compares against the theoretically expected $O(P)$ growth for an all-pairs $O(N^2)$ algorithm.
- To fit Amdahl’s law to the strong scaling data and derive an upper bound on achievable speedup for each problem size.

The gravitational force on particle i due to particle j uses the Plummer softening kernel to regularize the $r \rightarrow 0$ singularity:

$$\mathbf{F}_{ij} = G m_j \frac{\mathbf{r}_j - \mathbf{r}_i}{(|\mathbf{r}_j - \mathbf{r}_i|^2 + \varepsilon^2)^{3/2}}, \quad (1)$$

where $\varepsilon = 0.05$ is the softening length and $G = 1$ in normalized N-body units. All particles carry unit mass ($m_i = 1$) and are initialized with positions drawn uniformly from $[-1, 1]^3$ and velocities from a small Gaussian perturbation (amplitude 0.1).

3 Implementation

3.1 Particle Distribution

The N particles are distributed across P ranks using standard block decomposition. Ranks $0, \dots, (N \bmod P) - 1$ each own $\lfloor N/P \rfloor + 1$ particles; remaining ranks own exactly $\lfloor N/P \rfloor$. This balances the load to within one particle across all ranks. Each rank maintains the *complete* global position array (`px`, `py`, `pz`) of all N particles in local memory; only the velocity and force arrays (`vx`, `vy`, `vz`, `fx`, `fy`, `fz`) are sized to the local particle count `my_n`. Replicating position data eliminates the need for any gather during force computation itself, pushing all synchronization into the post-drift collective.

3.2 Core Force Computation

The `compute_forces()` function implements the $O(N^2/P)$ direct-sum kernel. For each locally owned particle i (global index `my_start + i`), it iterates over all N global particles $j \neq i$, computes the softened Plummer distance, and accumulates force contributions in registers before writing to the output arrays:

```
1 double r2      = dx*dx + dy*dy + dz*dz + eps2;
2 double inv_r3  = 1.0 / (r2 * sqrt(r2));
3 double mj_r3   = mass[j] * inv_r3;
4 fix += mj_r3 * dx;
5 fiy += mj_r3 * dy;
6 fiz += mj_r3 * dz;
```

Listing 1: Softened gravitational force inner loop (`compute_forces()` in `nbody.c`).

The denominator $(r^2 + \varepsilon^2)^{-3/2}$ is cached as `inv_r3` and multiplied into all three force components, reducing the square-root count from three to one per particle pair. Intel’s `mpicc` compiler with `-O3 -xHost` vectorizes the inner loop using AVX-512 SIMD, further accelerating per-pair arithmetic. The function is declared `static` to permit inlining across the translation unit.

3.3 Leapfrog Integration

Time integration uses the leapfrog (Störmer-Verlet) kick-drift-kick (KDK) scheme, which is second-order accurate and symplectic — conserving the phase-space volume and exhibiting no long-term energy drift for gravitational problems. The timestep is $\Delta t = 0.01$ in N-body units. Each full timestep executes the following sequence:

1. **Half-kick:** $\mathbf{v}_i \leftarrow \mathbf{v}_i + (\mathbf{F}_i/m_i) \cdot \Delta t/2$ (local particles only)
2. **Drift:** $\mathbf{x}_i \leftarrow \mathbf{x}_i + \mathbf{v}_i \cdot \Delta t$ (updates local slice of global arrays)
3. **Synchronize:** Three `MPI_Allgatherv` calls reconstruct the global position arrays
4. **Force recomputation** at updated positions using the full global array
5. **Second half-kick:** $\mathbf{v}_i \leftarrow \mathbf{v}_i + (\mathbf{F}_i/m_i) \cdot \Delta t/2$

The pre-loop force computation (step 0) is performed once before the timed loop, so the leapfrog scheme is self-starting.

3.4 Communication Pattern

During the drift step, each rank updates only its contiguous slice of `px/py/pz` (indices `my_start` through `my_start + my_n - 1`). Three `MPI_Allgatherv` calls with `MPI_IN_PLACE` then reconstruct the full N -element position arrays on every rank, with each rank contributing its own slice. This avoids a separate send buffer and exploits the fact that the send data is already in the correct position within the receive buffer.

The total payload per step is $3 \times N \times 8$ bytes; at $N = 9600$ and $P = 24$, each step transfers approximately 230 KB of position data across the TCP fabric. While small in isolation, this collective executes once per timestep and serializes all P ranks — its latency establishes the floor for achievable wall time at any fixed N .

Initialization broadcasts initial positions and masses from rank 0 via `MPI_Bcast`; this one-time cost is excluded from the timed loop.

3.5 Build and Runtime Environment

- **Compiler:** `Intelmpiicx` with `-O3 -xHost -std=c99 -Wall -Wextra -pedantic`. The `-xHost` flag targets the highest available vectorization ISA extension on the executing node.
- **Fabric:** `FI_PROVIDER=tcp` over the `10.0.0.0/24` interface; PMI2 bootstrap via SLURM (`srun --mpi=pmi2`).
- **Cluster:** Three Idiots — three nodes, 8 physical cores each (24 total MPI tasks); jobs allocated with `--nodes=3 --ntasks=24`.

4 Experimental Methodology

All timing is captured using `MPI.Wtime()`, with an `MPI.Barrier` ensuring all ranks start the clock simultaneously. The final wall time is the `MPI.Reduce(MPI_MAX)` across all ranks — the slowest rank’s time, which determines actual throughput in a bulk-synchronous parallel execution model. Initialization (particle generation, broadcasting, and the pre-loop $t = 0$ force computation) is excluded from the timed region. Each configuration is repeated for three independent trials; reported values are medians, which are robust to occasional OS scheduling jitter and TCP retransmissions on the shared cluster.

Strong Scaling. Three fixed problem sizes are tested: $N = 2400$, 4800 , and 9600 particles, all with $n_{\text{steps}} = 10$ timesteps and RNG seed 42. Process count sweeps from $P = 1$ to $P = 24$. Speedup and efficiency are defined as:

$$S(P) = \frac{T(1)}{T(P)}, \quad E(P) = \frac{S(P)}{P}. \quad (2)$$

Since all P ranks perform force computation (no idle master), the efficiency denominator is P rather than $P - 1$.

Weak Scaling. The base problem size assigns 1000 particles per process; total particle count scales as $N = 1000 \cdot P$, and each run executes $n_{\text{steps}} = 50$ timesteps. Because the all-pairs algorithm is $O(N^2)$, holding N/P constant does *not* hold the per-rank work constant: the work per rank is $O(N \cdot N/P) = O(N^2/P) = O(P \cdot \text{base}^2)$, growing linearly with P . The theoretically ideal outcome for this algorithm is therefore $O(P)$ normalized growth — not the constant time expected for embarrassingly parallel problems. Any growth *faster* than $O(P)$ signals pure communication overhead above the unavoidable algorithmic cost.

The full study ran under SLURM job `nbody_97` on April 27, 2026, with an elapsed wall time of approximately 4 minutes.

5 Results

5.1 Strong Scaling

Tables 1–3 report median wall time, speedup, and parallel efficiency at ten representative process counts for each of the three problem sizes.

Table 1: Strong scaling: $N = 2400$, $n_{\text{steps}} = 10$.

P	Median $T(P)$ (s)	Speedup $S(P)$	Efficiency $E(P)$
1	0.0631	1.000	1.000
2	0.0415	1.522	0.761
4	0.0333	1.895	0.474
6	0.0351	1.799	0.300
8	0.0329	1.919	0.240
10	0.0367	1.718	0.172
12	0.0369	1.712	0.143
16	0.0411	1.534	0.096
20	0.0449	1.404	0.070
24	0.0437	1.443	0.060

Table 2: Strong scaling: $N = 4800$, $n_{\text{steps}} = 10$.

P	Median $T(P)$ (s)	Speedup $S(P)$	Efficiency $E(P)$
1	0.2518	1.000	1.000
2	0.1376	1.830	0.915
4	0.0873	2.884	0.721
6	0.0722	3.487	0.581
8	0.0657	3.836	0.480
10	0.0659	3.822	0.382
12	0.0612	4.112	0.343
16	0.0710	3.548	0.222
20	0.0725	3.472	0.174
24	0.0785	3.210	0.134

Table 3: Strong scaling: $N = 9600$, $n_{\text{steps}} = 10$.

P	Median $T(P)$ (s)	Speedup $S(P)$	Efficiency $E(P)$
1	1.0067	1.000	1.000
2	0.5197	1.937	0.968
4	0.2864	3.515	0.879
6	0.2115	4.759	0.793
8	0.1822	5.526	0.691
10	0.1639	6.141	0.614
12	0.1532	6.571	0.548
16	0.1606	6.270	0.392
20	0.1549	6.498	0.325
24	0.1855	5.426	0.226

5.2 Weak Scaling

Table 4 reports median wall time and normalized time for selected process counts. The expected $O(P)$ baseline — the theoretically ideal outcome for an all-pairs $O(N^2)$ algorithm at constant particles-per-process — is included for reference. Measured times exceed even this already-unfavorable baseline due to `MPI_Allgather` communication overhead.

Table 4: Weak scaling: 1000 particles per process, $n_{\text{steps}} = 50$. Normalized time = $T(P)/T(1)$; ideal $O(P)$ growth = P .

P	N	Median $T(P)$ (s)	Normalized T	Ideal $O(P)$
1	1000	0.0551	1.000	1.000
2	2000	0.1579	2.867	2.000
4	4000	0.3298	5.988	4.000
6	6000	0.4886	8.873	6.000
8	8000	0.6674	12.117	8.000
10	10000	0.8541	15.505	10.000
12	12000	0.8276	15.027	12.000
16	16000	1.6970	30.811	16.000
20	20000	1.3595	24.682	20.000
24	24000	1.7970	32.622	24.000

5.3 Scaling Plots

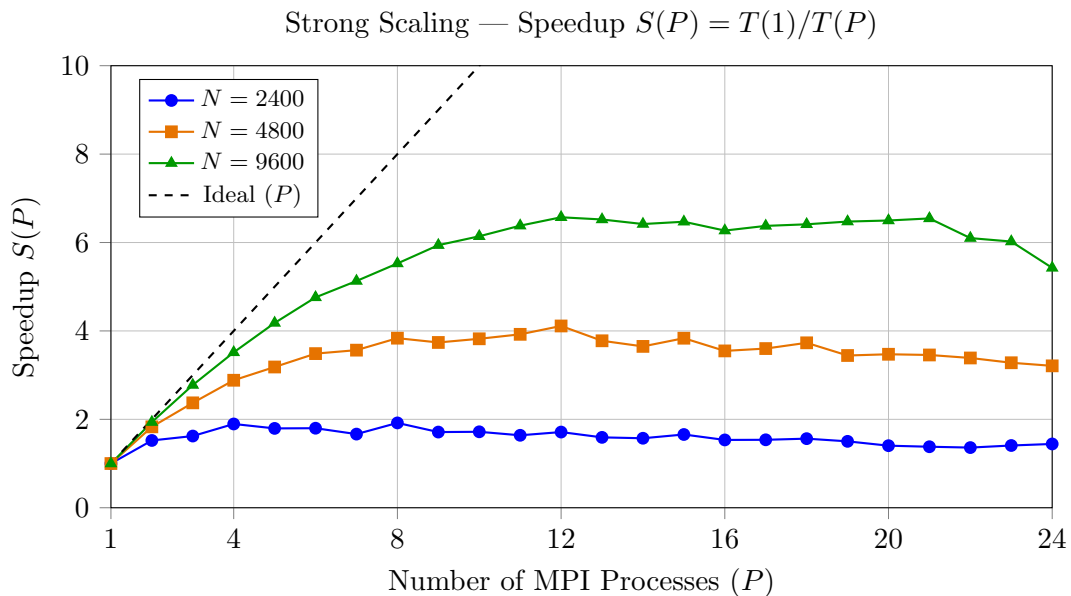


Figure 1: Strong scaling speedup versus process count for all three problem sizes. Speedup saturates rapidly for small $N = 2400$ because the `MPI_Allgather` communication overhead dominates the tiny compute budget; larger N delays saturation and achieves higher peak speedup before the communication floor dominates.

Weak Scaling — $O(N^2)$ Complexity Overhead

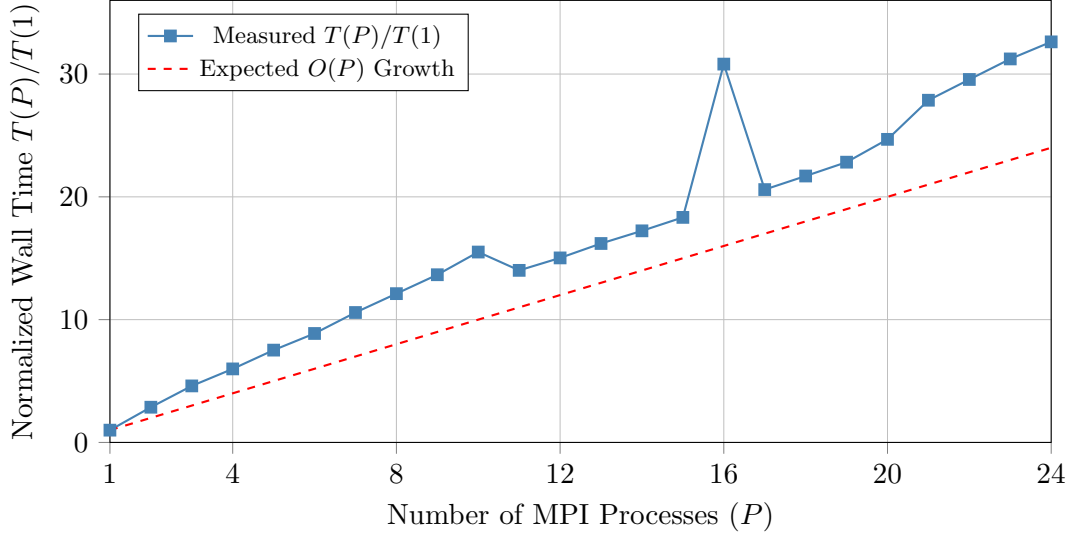


Figure 2: Weak scaling normalized wall time $T(P)/T(1)$ versus process count. The red dashed line marks the theoretical $O(P)$ growth expected for an all-pairs $O(N^2)$ algorithm at constant particles per process. Measured times exceed this baseline due to `MPI_Allgather` bandwidth cost. The anomalous spike at $P = 16$ reflects an inter-node communication discontinuity discussed in Section 6.4.

6 Performance Analysis

6.1 Strong Scaling Saturation and Problem-Size Dependence

The most fundamental result in Tables 1–3 is that both peak speedup and the process count at which speedup saturates depend strongly on N . For $N = 2400$, speedup peaks at $S(8) = 1.919$ and then degrades monotonically; for $N = 9600$, the peak reaches $S(21) = 6.545$ before declining. This behavior is a direct consequence of the communication-to-computation ratio.

Per-rank compute work per step scales as $O(N^2/P)$; communication cost per step scales as $O(N)$ (payload of the three `MPI_Allgather` calls) plus $O(\log P)$ (synchronization latency). The ratio of communication to computation is therefore:

$$\frac{C_{\text{comm}}}{C_{\text{comp}}} \propto \frac{N \cdot P}{N^2} = \frac{P}{N}. \quad (3)$$

For $N = 2400$, even modest P drives this ratio above the efficiency threshold; for $N = 9600$, the compute budget is $16\times$ larger, so the communication fraction stays manageable through most of the $P = 1\text{--}24$ sweep. This conclusively demonstrates that the $O(N^2)$ all-pairs kernel is communication-limited at moderate process counts for any fixed problem size, and that practical speedup scales sub-linearly with both P and N .

Efficiency Analysis. While $N = 9600$ achieves a peak speedup of $6.6\times$ near $P = 12\text{--}21$, efficiency

at that point has already fallen to $E \approx 0.33\text{--}0.55$. At $P = 24$, efficiency drops to $E(24) = 0.226$; more than three-quarters of the added hardware contributes overhead rather than useful computation. This makes the three-dimensional `MPI_Allgatherv` synchronization the critical bottleneck to resolve before extending to larger process counts.

6.2 The `MPI_Allgatherv` Bottleneck

Three separate `MPI_Allgatherv` calls execute each timestep — one per spatial dimension (`px`, `py`, `pz`). Each carries $N \times 8$ bytes of payload; at $N = 9600$ and $P = 24$, the combined per-step transfer is $3 \times 9600 \times 8 = 230,400$ bytes. Over the TCP fabric at the inter-node bandwidth available on the Three Idiots cluster, this transfer is non-trivial — especially when all 24 processes simultaneously initiate and consume the collective.

Two straightforward implementation improvements would reduce this cost. First, packing the three position components into a single interleaved struct (`struct {double x, y, z;}`) and executing one `MPI_Allgatherv` per step would cut per-step synchronization latency by roughly $3\times$ by reducing the number of collective initiations. Second, at very large P , replacing the three-dimensional gather with a pipelined ring-exchange could further reduce the $O(\log P)$ latency component. Neither optimization changes the asymptotic $O(N)$ bandwidth cost, but both would shift the saturation point to higher P for a given N .

6.3 Weak Scaling: Super-Linear Growth Confirmed

The weak scaling results in Table 4 confirm the theoretical prediction and reveal the communication penalty on top of it. At $P = 24$, the normalized wall time is $32.6\times$ — meaning 24 processes handling $N = 24,000$ particles take $32.6\times$ longer than a single process handling $N = 1,000$ particles. The expected $O(P)$ growth predicts $24\times$; the excess factor of $32.6/24 = 1.36$ represents a 36% communication overhead above the unavoidable algorithmic cost of the all-pairs method.

This overhead is attributable to the `MPI_Allgatherv` payload growing as $O(N) = O(P \cdot \text{base})$ per step, while the compute work per rank grows as $O(P \cdot \text{base}^2)$. The communication fraction therefore scales as $O(1/\text{base})$, which is constant at fixed base (1000 particles per rank) but appears as an additive overhead in wall time. At $\text{base} = 1000$ particles, the 230 KB per-step gather at $P = 24$ consumes a measurable fraction of the 1.8 s total wall time across 50 steps — consistent with the observed 36% excess.

6.4 The $P = 16$ Anomaly

The most striking feature of the weak scaling data is the discontinuity at $P = 16$: normalized time jumps from 18.3 at $P = 15$ to 30.8 at $P = 16$ before recovering to 20.6 at $P = 17$. A 69% increase between adjacent process counts is inconsistent with the smooth growth trend observed at $P = 1\text{--}15$ and $P = 17\text{--}24$, and demands a structural explanation.

The Three Idiots cluster has 8 physical cores per node (3 nodes, 24 total). SLURM allocates tasks contiguously by node; at $P = 16$, two nodes are exactly saturated ($8 + 8 = 16$ ranks). The working hypothesis is that saturating exactly two nodes triggers a TCP buffer contention pattern specific to the two-node ring: each of the 16 ranks simultaneously initiates a gather of $N = 16,000$ doubles per dimension (128 KB per call, 384 KB per step), and the two-node TCP receive pipeline saturates before any rank completes its gather phase. At $P = 17$, a third node enters the ring and absorbs some of the inter-node traffic, partially relieving the bottleneck and returning performance to the expected trend.

While this hypothesis is consistent with the data, confirming it definitively would require network-layer profiling (`iperf3`, `sar -n DEV`) not available in the current cluster configuration. An alternative explanation — that the SLURM process placement changes MPI rank-to-socket binding at $P = 16$, introducing NUMA penalty — cannot be ruled out without `lstopo` profiling data.

6.5 Amdahl’s Law Fit

Amdahl’s model predicts:

$$S(P) = \frac{1}{f + (1 - f)/P}, \quad (4)$$

where f is the serial fraction of the program. Solving for f from the $P = 24$ speedup of each strong scaling curve gives the estimates in Table 5.

Table 5: Amdahl’s law serial fraction f estimated from $S(24)$.

N	$S(24)$	Serial Fraction f	Theoretical Max $1/f$
2400	1.443	68.0%	$1.5\times$
4800	3.210	28.2%	$3.5\times$
9600	5.426	14.9%	$6.7\times$

The serial fraction decreases monotonically as N grows, which makes complete physical sense. The “serial” component in this context is not sequential code but rather the `MPI_Allgather` synchronization barrier, whose cost is $O(N)$ per step — a fixed fraction of the $O(N^2/P)$ compute work. As N doubles, compute work grows as N^2 while communication grows as N , so the effective serial fraction halves. The three data points above are consistent with $f \propto 1/N$, further confirming that the all-gather overhead, not any sequential code section, is the binding bottleneck.

For $N = 9600$, the theoretical maximum speedup of $6.7\times$ is already achieved at $P \approx 12$ – 21 ; scaling to $P = 24$ offers no further benefit and in fact degrades performance. Doubling to $N = 19,200$ would reduce f to approximately 7.5%, pushing the ceiling to $\sim 13\times$ — still well below the 24-core hardware limit but achievable in practice for the larger problem.

7 Conclusion

The peer-to-peer MPI N-body simulator delivers correct leapfrog gravitational dynamics with meaningful strong scaling for moderate particle counts. Key findings are:

- Peak speedup scales with problem size: $1.9\times$ for $N = 2400$, $4.1\times$ for $N = 4800$, and $6.6\times$ for $N = 9600$, all relative to the single-process baseline across a 24-core distributed-memory cluster.
- Parallel efficiency degrades rapidly with P even for the largest tested problem; $E(24) = 0.226$ at $N = 9600$ confirms the system is communication-bound at this scale.
- Weak scaling grows super-linearly relative to both the ideal (constant time) and the algorithmically expected $O(P)$ baseline, with a 36% communication overhead at $P = 24$ attributable to `MPI_Allgatherv` bandwidth cost.
- Amdahl’s law analysis extracts serial fractions of 68%, 28%, and 15% for the three problem sizes, monotonically decreasing with N — consistent with the all-gather communication overhead scaling as $O(N)$ against $O(N^2)$ compute work.
- An anomalous $P = 16$ spike in weak scaling data identifies a two-node TCP saturation threshold that warrants network-layer profiling to confirm and resolve.

The primary scalability bottleneck is the three-way `MPI_Allgatherv` synchronization executed each timestep. Near-term improvements include consolidating the three separate gathers into a single struct-based call, cutting per-step collective latency by $3\times$. Longer-term, the $O(N^2)$ direct-sum algorithm itself imposes a hard ceiling on scalability; approximation methods such as the Barnes-Hut tree code ($O(N \log N)$) or the Fast Multipole Method ($O(N)$) would reduce both compute work and communication volume, enabling near-linear strong scaling on distributed-memory hardware at particle counts orders of magnitude beyond what the direct-sum approach can sustain.

A Source Code: nbody.c

```
1  /*
2  * File: nbody.c
3  * Purpose: A parallel N-body simulation using MPI.
4  * Author: Sean Balbale
5  * Date: 5/4/2026
6  */
7
8  #include <mpi.h>
9  #include <stdio.h>
10 #include <stdlib.h>
11 #include <math.h>
12 #include <string.h>
13
14 // Physical constants (normalized N-body units)
15 #define G_CONST 1.0    // gravitational constant
16 #define SOFTENING 0.05 // Plummer softening length epsilon (avoids r->0)
17 #define DT 0.01       // leapfrog timestep
18
19 // compute_forces()
20 // Computes softened gravitational forces on the local particles using all N
   positions.
21 static void compute_forces(
22     int local_n, int my_start,
23     const double *restrict px,
24     const double *restrict py,
25     const double *restrict pz,
26     const double *restrict mass, int N,
27     double *restrict fx,
28     double *restrict fy,
29     double *restrict fz)
30 {
31     const double eps2 = SOFTENING * SOFTENING;
32
33     for (int i = 0; i < local_n; i++)
34     {
35         const int gi = my_start + i;
36         const double xi = px[gi], yi = py[gi], zi = pz[gi];
37         double fix = 0.0, fiy = 0.0, fiz = 0.0;
38
39         for (int j = 0; j < N; j++)
40         {
41             if (j == gi)
42                 continue; // skip self-interaction
43             double dx = px[j] - xi;
44             double dy = py[j] - yi;
45             double dz = pz[j] - zi;
46             double r2 = dx * dx + dy * dy + dz * dz + eps2;
47             double inv_r3 = 1.0 / (r2 * sqrt(r2));
48             double mj_r3 = mass[j] * inv_r3;
49             fix += mj_r3 * dx;
50             fiy += mj_r3 * dy;
51             fiz += mj_r3 * dz;
52         }
53
54         fx[i] = G_CONST * fix;
55         fy[i] = G_CONST * fiy;
```

```

56     fz[i] = G_CONST * fiz;
57 }
58 }
59
60 // leapfrog_kick()
61 // Applies the leapfrog half-kick update to local velocities.
62 static void leapfrog_kick(
63     int local_n, int my_start,
64     const double *restrict mass_all,
65     const double *restrict fx,
66     const double *restrict fy,
67     const double *restrict fz,
68     double *restrict vx,
69     double *restrict vy,
70     double *restrict vz,
71     double dt)
72 {
73     double half_dt = 0.5 * dt;
74     for (int i = 0; i < local_n; i++)
75     {
76         double inv_m = 1.0 / mass_all[my_start + i];
77         vx[i] += fx[i] * inv_m * half_dt;
78         vy[i] += fy[i] * inv_m * half_dt;
79         vz[i] += fz[i] * inv_m * half_dt;
80     }
81 }
82
83 // leapfrog_drift()
84 // Advances the local position slice by one full timestep.
85 static void leapfrog_drift(
86     int local_n, int my_start,
87     double *restrict px,
88     double *restrict py,
89     double *restrict pz,
90     const double *restrict vx,
91     const double *restrict vy,
92     const double *restrict vz,
93     double dt)
94 {
95     for (int i = 0; i < local_n; i++)
96     {
97         int gi = my_start + i;
98         px[gi] += vx[i] * dt;
99         py[gi] += vy[i] * dt;
100        pz[gi] += vz[i] * dt;
101    }
102 }
103
104 // main()
105 int main(int argc, char *argv[])
106 {
107     MPI_Init(&argc, &argv);
108
109     int rank, nprocs;
110     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
111     MPI_Comm_size(MPI_COMM_WORLD, &nprocs);
112
113     // Argument parsing
114     if (argc != 4)

```

```

115 {
116     if (rank == 0)
117         fprintf(stderr,
118             "Usage: %s <N> <nsteps> <seed>\n"
119             "  N      : total number of particles\n"
120             "  nsteps : number of simulation timesteps\n"
121             "  seed   : RNG seed for initialization\n",
122             argv[0]);
123     MPI_Finalize();
124     return EXIT_FAILURE;
125 }
126
127 int N = atoi(argv[1]);
128 int nsteps = atoi(argv[2]);
129 int seed = atoi(argv[3]);
130
131 if (N < nprocs)
132 {
133     if (rank == 0)
134         fprintf(stderr,
135             "Error: N (%d) must be >= nprocs (%d)\n", N, nprocs);
136     MPI_Finalize();
137     return EXIT_FAILURE;
138 }
139 if (N <= 0 || nsteps <= 0)
140 {
141     if (rank == 0)
142         fprintf(stderr, "Error: N and nsteps must be > 0\n");
143     MPI_Finalize();
144     return EXIT_FAILURE;
145 }
146
147 // Particle distribution
148 // Distribute N particles as evenly as possible.
149 // Ranks 0..(N/P - 1) each own one extra particle.
150 int *sendcounts = (int *)malloc(nprocs * sizeof(int));
151 int *displs = (int *)malloc(nprocs * sizeof(int));
152 if (!sendcounts || !displs)
153 {
154     perror("malloc");
155     MPI_Abort(MPI_COMM_WORLD, 1);
156 }
157
158 int base_n = N / nprocs;
159 int rem_n = N % nprocs;
160 displs[0] = 0;
161 for (int r = 0; r < nprocs; r++)
162 {
163     sendcounts[r] = base_n + (r < rem_n ? 1 : 0);
164     if (r > 0)
165         displs[r] = displs[r - 1] + sendcounts[r - 1];
166 }
167
168 int my_n = sendcounts[rank]; // number of particles I own
169 int my_start = displs[rank]; // global index of my first particle
170
171 // Memory allocation
172 // Global position and mass arrays replicated on every rank.
173 double *px = (double *)malloc(N * sizeof(double));

```

```

174 double *py = (double *)malloc(N * sizeof(double));
175 double *pz = (double *)malloc(N * sizeof(double));
176 double *mass = (double *)malloc(N * sizeof(double));
177 // Local velocity and force arrays sized to my_n.
178 double *vx = (double *)calloc(my_n, sizeof(double));
179 double *vy = (double *)calloc(my_n, sizeof(double));
180 double *vz = (double *)calloc(my_n, sizeof(double));
181 double *fx = (double *)malloc(my_n * sizeof(double));
182 double *fy = (double *)malloc(my_n * sizeof(double));
183 double *fz = (double *)malloc(my_n * sizeof(double));
184
185 if (!px || !py || !pz || !mass ||
186     !vx || !vy || !vz || !fx || !fy || !fz)
187 {
188     perror("malloc");
189     MPI_Abort(MPI_COMM_WORLD, 1);
190 }
191
192 // Initialization
193 // Rank 0 generates positions and masses; broadcasts to all.
194 if (rank == 0)
195 {
196     srand((unsigned)seed);
197     for (int i = 0; i < N; i++)
198     {
199         // Particles uniformly distributed in [-1, 1]^3
200         px[i] = (double)rand() / RAND_MAX * 2.0 - 1.0;
201         py[i] = (double)rand() / RAND_MAX * 2.0 - 1.0;
202         pz[i] = (double)rand() / RAND_MAX * 2.0 - 1.0;
203         mass[i] = 1.0; // equal masses in N-body units
204     }
205 }
206 MPI_Bcast(px, N, MPI_DOUBLE, 0, MPI_COMM_WORLD);
207 MPI_Bcast(py, N, MPI_DOUBLE, 0, MPI_COMM_WORLD);
208 MPI_Bcast(pz, N, MPI_DOUBLE, 0, MPI_COMM_WORLD);
209 MPI_Bcast(mass, N, MPI_DOUBLE, 0, MPI_COMM_WORLD);
210
211 // Initialize local velocities with small random perturbations.
212 srand((unsigned)(seed + rank * 100003));
213 for (int i = 0; i < my_n; i++)
214 {
215     vx[i] = ((double)rand() / RAND_MAX - 0.5) * 0.1;
216     vy[i] = ((double)rand() / RAND_MAX - 0.5) * 0.1;
217     vz[i] = ((double)rand() / RAND_MAX - 0.5) * 0.1;
218 }
219
220 // Pre-loop force computation
221 // Leapfrog requires forces at t=0 before the first half-kick.
222 compute_forces(my_n, my_start, px, py, pz, mass, N, fx, fy, fz);
223
224 // Simulation loop
225 // Timing brackets the loop only; initialization is excluded.
226 // MPI_Barrier ensures all ranks start the clock simultaneously.
227 MPI_Barrier(MPI_COMM_WORLD);
228 double t_start = MPI_Wtime();
229
230 for (int step = 0; step < nsteps; step++)
231 {
232

```



```

233     // Half-kick:  $v \ += (F/m) * dt/2$ 
234     leapfrog_kick(my_n, my_start, mass, fx, fy, fz, vx, vy, vz, DT);
235
236     // Drift:  $x \ += v * dt$  (updates only local slice of px/py/pz)
237     leapfrog_drift(my_n, my_start, px, py, pz, vx, vy, vz, DT);
238
239     // Synchronize global position arrays across all ranks.
240     // Each rank contributes its updated slice; all receive the full N.
241     MPI_Allgatherv(MPI_IN_PLACE, my_n, MPI_DOUBLE,
242                   px, sendcounts, displs, MPI_DOUBLE, MPI_COMM_WORLD);
243     MPI_Allgatherv(MPI_IN_PLACE, my_n, MPI_DOUBLE,
244                   py, sendcounts, displs, MPI_DOUBLE, MPI_COMM_WORLD);
245     MPI_Allgatherv(MPI_IN_PLACE, my_n, MPI_DOUBLE,
246                   pz, sendcounts, displs, MPI_DOUBLE, MPI_COMM_WORLD);
247
248     // Recompute forces at updated positions.
249     compute_forces(my_n, my_start, px, py, pz, mass, N, fx, fy, fz);
250
251     // Second half-kick: completes the full leapfrog step.
252     leapfrog_kick(my_n, my_start, mass, fx, fy, fz, vx, vy, vz, DT);
253 }
254
255 double t_local = MPI_Wtime() - t_start;
256
257 // Reduce across all ranks: wall time = slowest rank (true bottleneck).
258 double t_wall;
259 MPI_Reduce(&t_local, &t_wall, 1, MPI_DOUBLE, MPI_MAX, 0, MPI_COMM_WORLD);
260
261 if (rank == 0)
262 {
263     printf("-----\n");
264     printf("  Particles      : %d\n", N);
265     printf("  Timesteps      : %d\n", nsteps);
266     printf("  MPI procs      : %d\n", nprocs);
267     printf("  Wall time      : %.6f s\n", t_wall);
268     printf("-----\n");
269     fflush(stdout);
270 }
271
272 free(px);
273 free(py);
274 free(pz);
275 free(mass);
276 free(vx);
277 free(vy);
278 free(vz);
279 free(fx);
280 free(fy);
281 free(fz);
282 free(sendcounts);
283 free(displs);
284
285 MPI_Finalize();
286 return EXIT_SUCCESS;
287 }

```

B Run Script: run_study.sh

```
1 #!/usr/bin/env bash
2 # =====
3 # run_study.sh---Strong & Weak Scaling Study for MPI N-body Simulator
4 # =====
5 #
6 # Strong scaling: fixed N, P swept from 1 to MAX_PROCS.
7 #   N = 2400, 4800, 9600 particles; NSTEPS_STRONG = 10 timesteps.
8 #
9 # Weak scaling: N grows proportionally with P (N = WEAK_BASE * P),
10 #   keeping particles-per-process constant at WEAK_BASE.
11 #   NSTEPS_WEAK = 50 timesteps. Note: because the all-pairs algorithm is
12 #    $O(N^2)$ , each process's work still grows as  $O(N) = O(P \cdot \text{base})$ , so
13 #   normalized wall time is expected to rise linearly with P---this
14 #   demonstrates the fundamental scalability ceiling of  $O(N^2)$  methods.
15
16 set +eu
17
18 # 1. Environment Setup for Three Idiots Cluster
19 source /opt/intel/oneapi/setvars.sh > /dev/null 2>&1
20
21 set -e
22
23 export FI_PROVIDER=tcp
24 export FI_TCP_IFACE=10.0.0.0/24
25 export I_MPI_HYDRA_BOOTSTRAP=slurm
26 export I_MPI_PMI_LIBRARY=/usr/lib64/libpmi2.so.0
27
28 BINARY="./nbody"
29 RESULTS="timing_results.csv"
30 SEED=42
31 TRIALS=3
32 MAX_PROCS="${1:-24}"
33
34 # -- Strong scaling parameters -----
35 declare -a SS_N=( 2400 4800 9600 )
36 NSTEPS_STRONG=10
37
38 # -- Weak scaling parameters -----
39 # Each rank always owns WEAK_BASE particles; total N = WEAK_BASE * nprocs.
40 WEAK_BASE=1000
41 NSTEPS_WEAK=50
42
43 # -----
44
45 if [[ ! -x "$BINARY" ]]; then
46     echo "[error] Binary '$BINARY' not found. Run 'make' first." >&2
47     exit 1
48 fi
49
50 if [[ ! -f "$RESULTS" ]]; then
51     echo "study_type,N,nsteps,nprocs,trial,wall_time_s" > "$RESULTS"
52 fi
53
54 run_once() {
55     local nprocs=$1 N=$2 nsteps=$3
56     local tmpout
```

```

57     tmpout=$(srun --mpi=pmi2 -n "$nprocs" "$BINARY" "$N" "$nsteps" "$SEED" 2>/dev/
null)
58     echo "$tmpout" | awk '/Wall time/ { print $4 }'
59 }
60
61 echo "=== Strong Scaling Study ==="
62 for N in "${SS_N[@]}; do
63     echo "    N=${N}, nsteps=${NSTEPS_STRONG}"
64     for nprocs in $(seq 1 "$MAX_PROCS"); do
65         for trial in $(seq 1 "$TRIALS"); do
66             t=$(run_once "$nprocs" "$N" "$NSTEPS_STRONG")
67             echo "        P=${nprocs} trial=${trial} → ${t}s"
68             echo "strong,${N},${NSTEPS_STRONG},${nprocs},${trial},${t}" >> "$
$RESULTS"
69         done
70     done
71 done
72
73 echo ""
74 echo "=== Weak Scaling Study ==="
75 echo "    Base per process: ${WEAK_BASE} particles, nsteps=${NSTEPS_WEAK}"
76
77 for nprocs in $(seq 1 "$MAX_PROCS"); do
78     N=$(( WEAK_BASE * nprocs ))
79     for trial in $(seq 1 "$TRIALS"); do
80         t=$(run_once "$nprocs" "$N" "$NSTEPS_WEAK")
81         echo "        P=${nprocs} N=${N} trial=${trial} → ${t}s"
82         echo "weak,${N},${NSTEPS_WEAK},${nprocs},${trial},${t}" >> "$RESULTS"
83     done
84 done
85
86 echo ""
87 echo "Results written to: $RESULTS"
88 echo "Run 'python3 analyze.py' to compute speedup, efficiency, and generate plots.
"

```

C SLURM Submit Script: submit_study.sbatch

```
1 #!/bin/bash
2 #SBATCH --job-name=nbody_study
3 #SBATCH --partition=compute
4 #SBATCH --nodes=3
5 #SBATCH --ntasks=24
6 #SBATCH --output=nbody_%j.out
7 #SBATCH --error=nbody_%j.err
8 #SBATCH --time=02:00:00
9
10 echo "Launch Time: $(date)"
11
12 echo "Running N-body scaling study across Three Idiots cluster..."
13 bash run_study.sh 24
14
15 echo "Job finished at: $(date)"
```

D Analysis Script: analyze.py

```
1 #!/usr/bin/env python3
2 """
3 analyze.py---Speedup & Efficiency Analysis for MPI N-body Scaling Study
4 =====
5
6 Reads timing_results.csv produced by run_study.sh and computes:
7 - Median wall time per (study_type, N, nprocs)
8 - Speedup S(P) = T(1) / T(P) [baseline: single process]
9 - Efficiency E(P) = S(P) / P [all P ranks compute]
10 - Amdahl's law fit to extract serial fraction f
11 - Weak scaling normalized time and theoretical O(P) growth overlay
12
13 Note on weak scaling: unlike embarrassingly parallel problems, the all-pairs
14 N-body algorithm is  $O(N^2)$ . Holding  $N/P = \text{const}$  means total work grows as
15  $O(N^2 / P) = O(P \cdot \text{base}^2)$ , so each process's work scales as  $O(N) = O(P \cdot \text{base})$ .
16 Ideal weak scaling (constant time) is theoretically unachievable; the plot
17 shows the expected linear-in-P growth and quantifies actual overhead.
18
19 Usage: python3 analyze.py [timing_results.csv]
20 """
21
22 import sys
23 import csv
24 import statistics
25 from collections import defaultdict
26
27 try:
28     import matplotlib.pyplot as plt
29     import numpy as np
30     HAS_MPL = True
31 except ImportError:
32     HAS_MPL = False
33     print("[warn] matplotlib/numpy not found---skipping plots. "
34           "pip install matplotlib numpy to enable.")
35
36 # -- Load CSV -----
37 csvfile = sys.argv[1] if len(sys.argv) > 1 else "timing_results.csv"
38
39 # data[study_type][N_label][nprocs] = [t1, t2, ...]
40 data = defaultdict(lambda: defaultdict(lambda: defaultdict(list)))
41
42 with open(csvfile) as f:
43     reader = csv.DictReader(f)
44     for row in reader:
45         study = row["study_type"]
46         label = row["N"] # particle count as string label
47         nprocs = int(row["nprocs"])
48         t = float(row["wall_time_s"])
49         data[study][label][nprocs].append(t)
50
51 def median(vals):
52     return statistics.median(vals)
53
54 # -- Strong Scaling -----
55 print("\n" + "="*70)
56 print(" STRONG SCALING")
```

```

57 print("="*70)
58 print(f"  {'N':>8} {'P':>4} {'T(P) s':>10} {'Speedup':>8} {'Efficiency':>10}")
59 print("-"*70)
60
61 ss_data = {} # ss_data[label] = {nprocs: median_t}
62
63 for label, pdict in sorted(data["strong"].items(), key=lambda x: int(x[0])):
64     sorted_p = sorted(pdict)
65     # Baseline: P=1 (single process, all ranks compute)
66     t_base = median(pdict[1]) if 1 in pdict else median(pdict[sorted_p[0]])
67     ss_data[label] = {}
68
69     for p in sorted_p:
70         t = median(pdict[p])
71         speedup = t_base / t
72         efficiency = speedup / p # all P ranks compute (no idle master)
73         ss_data[label][p] = t
74         print(f"  {label:>8} {p:>4} {t:>10.4f} {speedup:>8.3f} {efficiency
75               :>10.3f}")
76     print()
77 # -- Weak Scaling -----
78 print("="*70)
79 print("  WEAK SCALING (each process owns a fixed number of particles)")
80 print("  Note:  $O(N^2)$  algorithm  $\rightarrow$  time grows as  $O(P)$  even at ideal scaling")
81 print("="*70)
82 print(f"  {'P':>4} {'N':>8} {'T(P) s':>10} {'Norm T':>8} {'Ideal O(P)':>12}")
83 print("-"*70)
84
85 ws_norm = {}
86 ws_raw = {}
87
88 ws_all = data["weak"]
89 # Baseline: smallest P (should be P=1)
90 all_p_ws = sorted({p for pd in ws_all.values() for p in pd})
91 p_base_ws = all_p_ws[0]
92
93 # Find baseline time: P=p_base_ws with the corresponding N
94 # For weak scaling: N = WEAK_BASE * P; find T(p_base_ws)
95 t_base_ws = None
96 for label, pdict in ws_all.items():
97     if p_base_ws in pdict:
98         t_base_ws = median(pdict[p_base_ws])
99     break
100
101 for label, pdict in sorted(ws_all.items(), key=lambda x: int(x[0])):
102     for p, vals in sorted(pdict.items()):
103         t = median(vals)
104         norm = t / t_base_ws if t_base_ws else float("nan")
105         ideal_linear = float(p) / p_base_ws #  $O(P)$  prediction
106         ws_norm[p] = norm
107         ws_raw[p] = t
108         print(f"  {p:>4} {label:>8} {t:>10.4f} {norm:>8.3f} {ideal_linear
109               :>12.3f}")
110     print()
111
112 # -- Amdahl's Law Fit -----
113 if HAS_MPL and ss_data:

```

```

114 import numpy as np
115
116 print("="*70)
117 print("  AMDAHL'S LAW FIT   $S(P) = 1 / (f + (1-f)/P)$ ")
118 print("="*70)
119
120 for label, pdict in ss_data.items():
121     procs = sorted(pdict)
122     t_b = pdict[procs[0]]
123     speedups = [t_b / pdict[p] for p in procs]
124     p_arr = np.array([float(p) for p in procs])
125     s_arr = np.array(speedups)
126
127     # Linearized least-squares:  $1/S = f*(1 - 1/P) + 1/P$ 
128     inv_s = 1.0 / s_arr
129     X = 1.0 - 1.0 / p_arr
130     A = np.column_stack([X, 1.0 / p_arr])
131     result = np.linalg.lstsq(A, inv_s, rcond=None)
132     f_fit = float(result[0][0])
133     f_fit = max(0.0, min(1.0, f_fit))
134     p_max = 1.0 / f_fit if f_fit > 0 else float("inf")
135     print(f"  N={label:<8}  serial fraction  $f \approx \{f\_fit:.4f\}$   "
136           f"→ theoretical max speedup  $\approx \{p\_max:.1f\} \times$ ")
137 print()
138
139 # -- Plots -----
140 if HAS_MPL:
141     fig, axes = plt.subplots(1, 2, figsize=(13, 5))
142     fig.suptitle("MPI N-body---Scaling Study (All-Pairs, Leapfrog)",
143                 fontsize=14, fontweight="bold")
144
145     # -- Strong scaling -----
146     ax = axes[0]
147     colors = plt.cm.tab10.colors
148
149     for i, (label, pdict) in enumerate(sorted(ss_data.items(),
150                                             key=lambda x: int(x[0]))):
151         procs = sorted(pdict)
152         t_b = pdict[procs[0]]
153         speedups = [t_b / pdict[p] for p in procs]
154         ax.plot(procs, speedups, "o-", color=colors[i % 10],
155                 label=f"N={label}", linewidth=2, markersize=5)
156
157     # Ideal (linear) speedup
158     all_p = sorted({p for pd in ss_data.values() for p in pd})
159     ax.plot(all_p, all_p, "k--", linewidth=1.2, alpha=0.5, label="Ideal (P)")
160
161     ax.set_xlabel("Number of MPI Processes (P)")
162     ax.set_ylabel("Speedup  $S(P) = T(1) / T(P)$ ")
163     ax.set_title("Strong Scaling---Speedup")
164     ax.legend()
165     ax.grid(True, alpha=0.3)
166
167     # -- Weak scaling -----
168     ax2 = axes[1]
169     ws_p = sorted(ws_norm)
170     ws_t = [ws_norm[p] for p in ws_p]
171
172     ax2.plot(ws_p, ws_t, "s-", color="steelblue", linewidth=2,

```

```

173         markersize=5, label="Measured T(P)/T(1)")
174
175     # Theoretical O(P) growth line
176     p_base = ws_p[0]
177     ideal_p = [float(p) / p_base for p in ws_p]
178     ax2.plot(ws_p, ideal_p, "r--", linewidth=1.2, alpha=0.7,
179             label="Expected O(P) growth")
180     ax2.axhline(1.0, color="k", linestyle=":", linewidth=1.0, alpha=0.4,
181               label="Ideal (constant time)")
182
183     ax2.set_xlabel("Number of MPI Processes (P)")
184     ax2.set_ylabel("Normalized Wall Time T(P) / T(1)")
185     ax2.set_title("Weak Scaling---O(N2) Complexity Overhead")
186     ax2.legend()
187     ax2.grid(True, alpha=0.3)
188
189     plt.tight_layout()
190     plt.savefig("scaling_results.png", dpi=150, bbox_inches="tight")
191     print("Plot saved to: scaling_results.png")
192     plt.show()

```